

Acceptance Testing 2.2: Number Guessing Game

To do: Make a submission

To do: Receive a grade

Due: Tuesday, 29 October 2024, 10:00 AM

Overview

In this acceptance testing assignment, you will continue writing acceptance tests for the number guessing game application using Robot Framework. This time, you will focus on validating the scoreboard functionality and testing some mid-game scenarios using Robot Framework. The game still has the same structure which involves choosing a difficulty level, placing bets, and guessing numbers. The top ten high scores are saved to a scoreboard.

Start by downloading the game source code, which is attached to this assignment (see below).

Game Rules

- You start with 100 bet points.
- Choose a difficulty level:
 - Easy: 10 guesses
 - Hard: 5 guesses
- Place a bet with your bet points for each round.
- Guess the number between 1 and 100:
 - If you guess correctly, you can choose to double your prize or take your winnings.
 - If you choose to double, you have a 50% chance to either double your prize or lose it.
- If you run out of guesses, you can try to save your bet:
 - You have a 50% chance to keep your bet points or lose the bet points you placed for that round.
- After each round, you can decide to continue playing or finish the game and save your score to the scoreboard.
- The game ends when you choose to finish or you have no bet points left.
- The high scores are saved and displayed on the scoreboard. Try to get your name in the top 10!

How to Play the Game

Web Version:

- Run the Web Application:
 - Open a terminal and navigate to the project directory.
 - Run the following command: `python app.py`
 - Open your web browser and go to `http://127.0.0.1:5000/`.

CLI Version:

- Run the CLI Application:
 - Open a terminal and navigate to the project directory.
 - Run the following command: `python game.py`
 - Follow the prompts:
 - Enter your name.
 - Choose a difficulty level.
 - Place bets and guess numbers as prompted in the terminal.

How to Install Dependencies

- Ensure you have Python installed on your system.
- Open a terminal and navigate to the project directory.
- Run the following command to install the required dependencies:

```
pip install -r requirements.txt
```

Part 1: Scoreboard Verification

In this part, you will write acceptance tests to verify that the scoreboard updates correctly when new high scores are added. The tests will validate whether the scoreboard behaves as expected when fewer than 10 scores exist, exactly 10 scores exist, and when a new score either replaces or does not replace the lowest score.

Scenarios to Implement

- Scoreboard Update With Less Than 10 Scores**
 - Stimulus: The scoreboard currently has fewer than 10 scores.
 - Response: The scoreboard displays all existing scores.
 - Stimulus: A new player finishes the game with a high score.
 - Response: The game updates the bet points accordingly.
 - Stimulus: The user decides to finish the game.
 - Response: The game saves the score and displays the correctly updated scoreboard including the new score.
- Scoreboard Update With Exactly 10 Scores**
 - Stimulus: The scoreboard currently has exactly 10 scores.
 - Response: The scoreboard displays the top 10 scores.
 - Stimulus: A new player finishes the game with a score higher than the lowest on the scoreboard.
 - Response: The game accepts the correct guess, and asks to double.
 - Stimulus: The user chooses to take the award without doubling.
 - Response: The game updates the bet points accordingly.
 - Stimulus: The user decides to finish the game.
 - Response: The scoreboard updates by replacing the lowest score with the new high score.
- Scoreboard Not Updated with Exactly 10 Scores**
 - Stimulus: The scoreboard currently has exactly 10 scores.
 - Response: The scoreboard displays the top 10 scores.
 - Stimulus: A new player finishes the game with a score lower than the lowest on the scoreboard.
 - Response: The game accepts the correct guess, and asks to double.
 - Stimulus: The user chooses to take the award without doubling.
 - Response: The game updates the bet points accordingly.
 - Stimulus: The user decides to finish the game.
 - Response: The game does not update the scoreboard.

Part 2: Regular Game Path Scenarios with Set Session Parameters

In this part, you will write acceptance tests for regular game paths using Set Session Parameters to jump into specific parts of the game. You may already be familiar with these scenarios from the previous assignment, where you tested full game playthroughs with correct and incorrect guesses. This time, you will focus on testing similar scenarios, but starting from a mid-game state using Set Session Parameters keyword to set the game state to mid-game, specifying bet points, remaining guesses, and any other necessary parameters. This allows testing of particular functionalities without going through all previous steps. Don't focus on the scoreboard in this part.

Scenarios to Implement

- Winning the Game From Mid Game State**
 - Stimulus: The user is in the middle of a game with a specific number of bet points and guesses remaining.
 - Response: The game reflects the set session parameters accurately.
 - Stimulus: The player makes the correct guess to win the game.
 - Response: The game accepts the correct guess, and asks to double.
 - Stimulus: The user chooses to take the award without doubling.
 - Response: The game updates the bet points accordingly.
 - Stimulus: The user decides to finish the game.
 - Response: The game shows a "Congratulations" message.
- Losing the Game From Mid Game State**
 - Stimulus: The user is in the middle of a game with a specific number of bet points and one last guess remaining. Also, the user placed all the bets they have.
 - Response: The game reflects the set session parameters accurately.
 - Stimulus: The user makes an incorrect guess.
 - Response: The game displays an "Out of Guesses" message.
 - Stimulus: The user doesn't attempt to save the bet.
 - Response: The game updates the bet points accordingly, and shows a "Game Over" message.

Hints

1. In Robot Framework, always use 4 spaces for indentation (not tabs) to ensure your tests run correctly.

2. You can include keywords from `helpers.robot` using the Resource setting.

3. Set Session Parameters Helper Keyword

Utilize the Set Session Parameters helper keyword defined in `helpers.robot` to navigate directly to specific parts of the game, bypassing earlier steps. This is useful for efficiently testing invalid inputs without starting from the beginning each time.

Example Usage: Set Session Parameters TestUser1 easy 5 50 50 10

4. Scoreboard Modification during tests

In Part 1, there may be times when you need to manipulate the scoreboard to simulate different game scenarios (e.g., fewer than 10 scores, exactly 10 scores, etc.). Directly modifying the scoreboard is not the most ideal approach, we have provided keywords to back up the original scoreboard and temporarily modify it during your tests. You can check the `helpers.robot` file. If you lose the original scoreboard, you can get it from assignment files in MyCourses.

Backup Original Scoreboard File:

The keyword saves a copy of the current scoreboard. You need to pass the paths of both the scoreboard file and the backup file as arguments.

Example Usage:

Backup Original Scoreboard File app/scoreboard.txt ./tests/backup_scoreboard.txt

Overwrite Scoreboard File With Custom Data:

The method allows you to overwrite the scoreboard with test data, such as predefined scores. You need to pass the path of both the scoreboard file and custom content as arguments.

Example Usage:

Overwrite Scoreboard File With Custom Data app/scoreboard.txt alice:1500:hard\nbob:1200:easy

Restore Original Scoreboard File:

The keyword restores the original scoreboard so the game returns to its default state. You pass the path of the backup file and the original scoreboard file to restore the data.

Example Usage:

Restore Original Scoreboard File ./tests/backup_scoreboard.txt app/scoreboard.txt

Other Resources:

- Tutorial Exercises:** [Efcode Academy RF Katas](#)
- Introduction Slides:** [Robot Framework Introduction by Pekka Klärck](#)
- Official Documentation:** [Robot Framework Documentation](#)
- Getting Started with Installation:** [Robot Framework Installation Guide](#)
- Additional Learning and Tutorials:** [Robot Framework Learning Resources](#)

Before Submitting

Execute the tests and verify that they pass without errors.

Debug and fix any issues that you had during the test execution.

Ensure your tests use the provided keywords, follow the scenarios described in the assignment, and are properly structured. Use the Set Session Parameters keyword to navigate directly to specific parts of the game when needed. Use the Scoreboard Modification keywords to manipulate the scoreboard appropriately during your tests.

Submission Instructions:

For this assignment, you will submit two test files, one for each part of the assignment.

Part1: Implement your test cases into the `studentnumber-part1.robot` file. This file should include test cases for **Scoreboard Update with Less Than 10 Scores** (scenario1), **Scoreboard Update with Exactly 10 Scores** (scenario2), and **Scoreboard Not Updated with Exactly 10 Scores** (scenario3).

Part2: Implement your test cases into the `studentnumber-part2.robot` file. This file should include the test cases for **Winning the Game From Mid Game State** (scenario1), and **Losing the Game From Mid Game State** (scenario2). For each step, ensure the game provides appropriate feedback for invalid inputs and does not proceed to the next step.

Submission: File Submission.submit either two separate test files or a zip file containing the entire project folder.

Grading:

- 5-4 points:** The assignment is clear and well-organized. Test cases are correctly implemented, covering both normal and edge cases. Tests include appropriate validation of game responses. Set Session Parameters keyword is effectively used to simulate mid-game states, and the tests run without errors. Scoreboard modification is done correctly. The readability of the tests is high, with clear and concise steps.
- 3-2 points:** The assignment is mostly clear, but some key scenarios or edge cases are missing. Test cases are implemented with minor gaps, and response validation may be incomplete. Set Session Parameters and scoreboard modification keywords are used, but not with sensible arguments. The readability of the tests is adequate, but some parts may be unclear or difficult to follow.
- 1-0 points:** The assignment is incomplete or lacks clarity. Test cases are missing, incorrect, or fail to validate responses properly. The Set Session Parameters and scoreboard modification keywords are either not used or applied wrong, and the tests may not run correctly. The readability of the tests is unclear with disorganized steps.

Use of AI and Collaboration with Other Students:

Using AI tools to produce an answer for this assignment is not permitted. All parts of the submission must be produced by yourself (apart from files provided in the assignment). Collaboration with other students is not permitted. This is an individual assignment that you must author indepen dently.

AcceptanceTesting2-2.zip

14 October 2024, 4:22 PM

Submission status

Attempt number	This is attempt 1.
Submission status	No submissions have been made yet
Grading status	Not marked
Time remaining	Assignment is overdue by: 134 days 3 hours
Last modified	-

Previous activity

← Acceptance Testing 2.1: Number Guessing Game

Next activity

Acceptance testing 3: Oscar →

MyCourses support for students

A!

Aalto-yliopisto
Aalto-universitetet
Aalto University

Students

- MyCourses instructions for students
- Support form for students

Teachers

- MyCourses help
- MyTeaching Support

About service

- MyCourses protection of privacy
- Privacy notice
- Service description
- Accessibility summary

Nguyen Binh (Log out)